CLOUDLOG

Documentação Completa da Arquitetura

Equipe

Camilla Augusta Uber
João Davi
Laura Guillarducci

Versão 1.0

Arquitetura de Software e Soluções Cloud · PUCPR
Prof. Manoel Valerio da Silveira Neto
Outubro de 2026
Repositório: github.com/laducci/cloudlog-pjbl

1. Introdução e Metas 3
1.1 Objetivos de negócio 3
1.2 Requisitos funcionais relevantes 3
1.3 Partes interessadas 3
1.4 Objetivos de qualidade prioritários 4
2. Restrições Arquiteturais 5
3. Contexto e Escopo 6
3.1 Contexto de negócio 6
3.2 C4 Model - Nível 1: Contexto do Sistema 7
3.3 Contexto técnico e interfaces 7
4. Estratégia de Solução 9
4.1 Software Architecture Canvas 10
4.2 Estilos arquiteturais adotados 10
5. Visão de Blocos de Construção 11
5.1 C4 Model - Nível 2: Contêineres 11
5.2 C4 Model - Nível 3: Componentes do Serviço de Entregas 14
5.3 C4 Nível 4 / UML - Diagrama de Classes 16
5.4 C4 Nível 4 / UML - Diagrama de Componentes 19
5.5 Diagrama de Entidades e Relacionamentos (DER) 20
5.6 Clean Architecture e Vertical Slice 21
6. Visão de Execução 23
6.1 C4 Nível 4 / UML - Diagramas de Sequência 23
6.2 Cenário - rastreamento e alerta de desvio 26
6.3 Cenário - consulta do dashboard 27
6.4 Catálogo de eventos 28
7. Visão de Implantação 29
8. Conceitos Transversais 31
8.1 Testes unitários de arquitetura 31
9. Decisões Arquiteturais 33
10. Requisitos de Qualidade 34
10.1 Rastreabilidade entre requisitos e arquitetura 34
11. Riscos e Dívidas Técnicas 35

1. Introdução e Metas

O CloudLog é uma plataforma baseada em nuvem para gestão de operações de transporte rodoviário de cargas. O sistema centraliza o acompanhamento da frota, o planejamento e execução de rotas, o rastreamento em tempo real por GPS/telemetria, a comprovação digital das entregas, a geração de alertas de atraso ou desvio e a consolidação de indicadores operacionais.

O problema de negócio que motiva a solução é a baixa visibilidade da operação em tempo real, somada a falhas na gestão de rotas, atrasos na comunicação entre a central e os motoristas e dependência de comprovantes de entrega em papel. A arquitetura deve permitir que diferentes perfis acessem informações adequadas às suas responsabilidades, sem concentrar toda a solução em uma única interface ou componente.

Esta versão 1.0 completa a documentação parcial (v0.1). O provedor de nuvem foi definido (Microsoft Azure), a decomposição passou a seguir os estilos Serverless, Microfrontend, Backend for Frontend com Microservices e Database per Service, API Gateway e Arquitetura Orientada a Eventos, e o código entregue do Serviço de Entregas aplica Clean Architecture, Vertical Slice e testes unitários de arquitetura. Ao longo do documento, cada elemento é indicado como **implementado** (existe no código do repositório), **parcial** ou **planejado** (arquitetura-alvo).

1.1 Objetivos de negócio

- Reduzir custos operacionais de transporte em até 15%.
- Reduzir o tempo de atraso nas entregas em 25%.
- Digitalizar 100% da comprovação de entregas, reduzindo papel e retrabalho manual.
- Aumentar a rastreabilidade da frota e a velocidade de resposta da central logística.

1.2 Requisitos funcionais relevantes

ID	Requisito	Descrição	Usuário principal	Situação na v1.0
RF-01	Autenticação e Controle de Acesso	Login seguro, suporte a MFA e controle de acesso baseado em perfis/permissões (RBAC).	Todos	**Planejado**
RF-02	Rastreamento em Tempo Real	Monitoramento da geolocalização dos veículos por telemetria/GPS.	Gerente de Logística	**Planejado**
RF-03	Gestão de Rotas e Entregas	Criação, edição, pesquisa, exclusão e atribuição de rotas e entregas aos motoristas.	Gerente de Logística	**Implementado** (entregas)
RF-04	Comprovação Digital de Entrega	Envio de fotos de comprovantes e assinatura digital pelo aplicativo mobile.	Motorista	**Planejado**
RF-05	Alertas de Atraso/Desvio	Geração automática de alertas para a central quando houver atraso ou desvio de rota.	SAC / Gerente	**Parcial** (atraso)
RF-06	Dashboard de Desempenho	Relatórios consolidados de tempo de rota, consumo e eficiência.	Gerente de Logística	**Parcial** (tela)

1.3 Partes interessadas

Stakeholder	Expectativas principais
Gerente de Logística	Visibilidade da frota, redução de custos, criação de rotas, acompanhamento de entregas e indicadores.
Motorista de Frota	Aplicativo simples para consultar rotas, atualizar status e registrar comprovantes digitais.
Atendente de SAC	Consulta rápida de rastreamento e ocorrências para responder clientes.
Equipe de Desenvolvimento	Arquitetura modular, testável e evolutiva, com responsabilidades bem delimitadas.

Stakeholder	Expectativas principais
Operação de TI	Disponibilidade, observabilidade, segurança, backup e capacidade de suporte.
Gestão da empresa	Redução de custos, melhoria de SLA e dados confiáveis para tomada de decisão.

1.4 Objetivos de qualidade prioritários

ID	Objetivo	Descrição
Q1	Disponibilidade	A plataforma deve permanecer operacional durante a execução das rotas e permitir recuperação rápida em caso de falha.
Q2	Segurança	Autenticação forte, autorização por perfil, proteção de dados e trilhas de auditoria.
Q3	Desempenho	Posições recentes devem estar disponíveis em poucos segundos para a central.
Q4	Usabilidade	O fluxo de atualização e comprovação de entrega no mobile deve exigir poucas etapas.
Q5	Escalabilidade	O volume de eventos de telemetria deve crescer sem exigir redesenho completo da solução.
Q6	Manutenibilidade e testabilidade	Funcionalidades isoladas em fatias verticais e regras de dependência verificadas por testes automatizados (novo na v1.0).

2. Restrições Arquiteturais

As restrições abaixo condicionam a solução nesta versão. Em relação à v0.1, a restrição de fornecedor indefinido foi substituída pelas restrições concretas do ambiente usado na disciplina (Azure for Students, Azure Functions, Static Web Apps e MongoDB Atlas).

Tipo	Restrição	Impacto arquitetural
Técnica	A solução deve ser baseada em nuvem.	Acesso distribuído, elasticidade e alta disponibilidade fazem parte do objetivo da plataforma.
Técnica	Aplicação Web para logística/SAC e aplicativo Mobile para motoristas.	Perfis e contextos de uso diferentes exigem interfaces separadas.
Técnica	Integração com GPS/telemetria/IoT.	RF-02 depende de dados de localização provenientes de dispositivos externos.
Técnica	MFA e RBAC.	RF-01 estabelece autenticação reforçada e autorização baseada em perfil.
Técnica	Armazenamento de arquivos digitais.	RF-04 requer fotos e assinaturas, inadequadas como blobs no banco transacional.
Técnica	Microsoft Azure com assinatura Azure for Students.	Prioriza serviços gratuitos ou por consumo: Static Web Apps, Functions (Consumption), Service Bus Basic.
Técnica	Backend serverless em Azure Functions (Node.js, modelo v4).	Funções sem estado, inicialização rápida e reuso de conexões.
Técnica	Funções gerenciadas do Static Web Apps aceitam apenas gatilhos HTTP.	O consumidor de fila (ServiceBusQueueTrigger) roda em uma Function App dedicada.
Técnica	Serviço de Entregas persistido no MongoDB Atlas (cluster M0, Student Pack).	Modelo de documentos; cluster gratuito pausa após período sem uso.
Organizacional	Equipe de três alunos (Camilla Augusta Uber, João Davi, Laura Guillarducci) e prazo acadêmico.	Entregas incrementais; parte da arquitetura é documentada como alvo.
Organizacional	Documentação em arc42 (11 seções), C4 Model níveis 1 a 4, UML, DER e Software Architecture Canvas, em PDF.	Diagramas versionados como código no repositório.
Operacional	Conectividade móvel pode ser instável.	O aplicativo deve tratar falhas transitórias e sincronização posterior.
Legal/Segurança	Dados pessoais e evidências de entrega devem ser protegidos.	Acesso mínimo necessário, criptografia e rastreabilidade devem ser considerados.

3. Contexto e Escopo

Esta seção delimita a fronteira do CloudLog e identifica pessoas, sistemas externos e interfaces de comunicação. O C4 Nível 1 representa a solução como uma caixa preta, sem expor sua decomposição interna.

3.1 Contexto de negócio

Elemento externo	Entradas para o CloudLog	Saídas do CloudLog
Gerente de Logística	Rotas, atribuições, consultas e decisões operacionais.	Status, localização, alertas e indicadores.
Motorista de Frota	Atualizações de entrega, fotos, assinatura e ocorrências.	Rotas atribuídas, dados da entrega e confirmações.
Atendente de SAC	Consultas sobre entregas e ocorrências.	Status, localização e informações de atraso.
GPS / IoT	Eventos de localização e telemetria.	Sem saída de negócio obrigatória nesta versão.
Serviço de Notificações	Confirmação de entrega técnica do alerta.	Recebe solicitações de push/e-mail/SMS.
Provedor de Identidade (novo)	Identidade e tokens do usuário autenticado com MFA.	Pedidos de autenticação (OIDC).

3.2 C4 Model - Nível 1: Contexto do Sistema

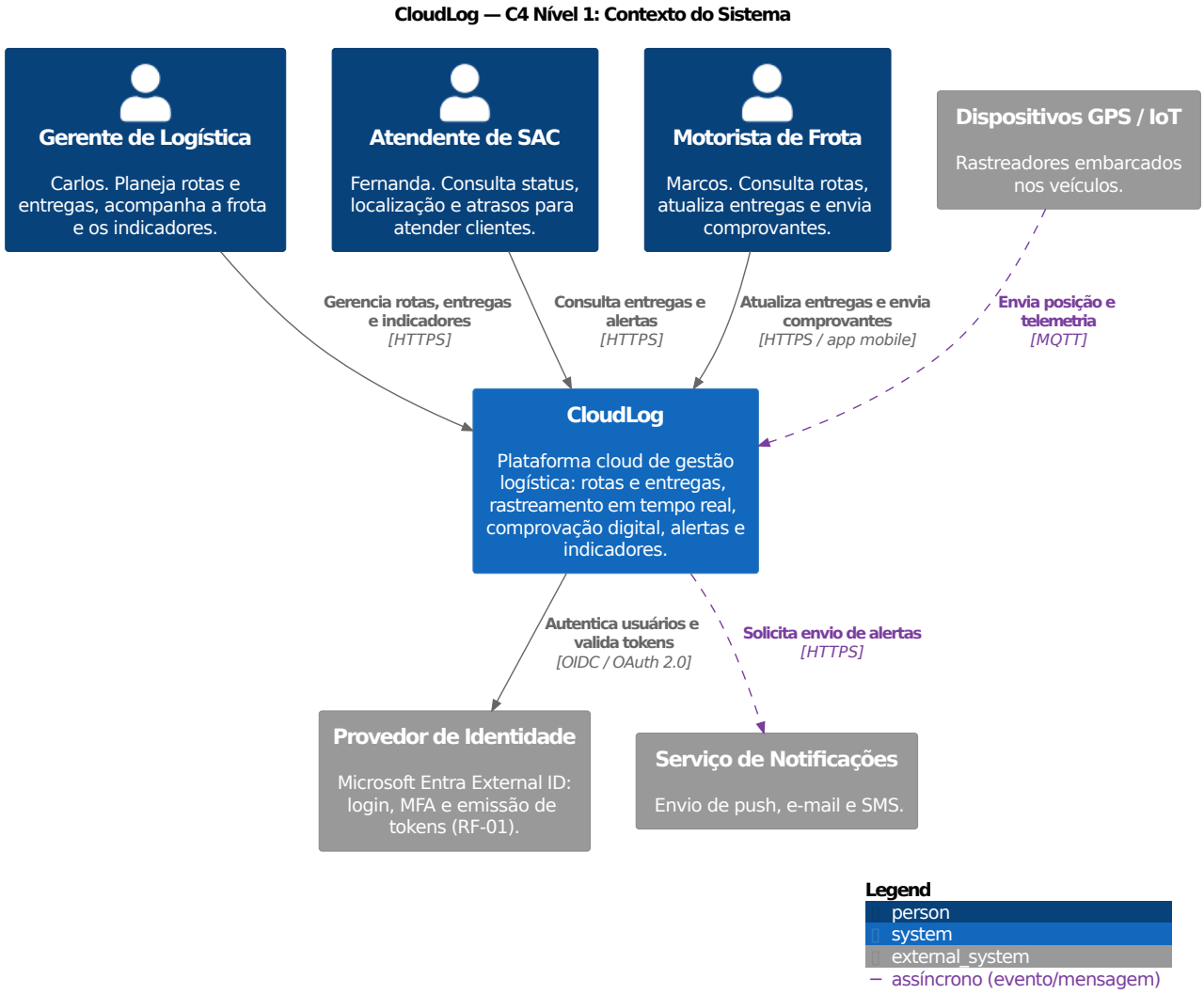


Figura 1 - C4 Nível 1: contexto do sistema CloudLog.

O diagrama separa claramente o sistema em estudo dos elementos externos. Gerente, motorista e SAC são pessoas; GPS/IoT, Provedor de Identidade e Serviço de Notificações são sistemas externos. Setas tracejadas em roxo indicam comunicação assíncrona. Nenhum detalhe de aplicação, API ou banco é mostrado neste nível.

3.3 Contexto técnico e interfaces

Interface	Canal	Finalidade	Situação
Aplicações cliente -> CloudLog	HTTPS (hoje `/api/*` no Static Web Apps; alvo: API Gateway)	Operações de negócio, autenticação e consultas.	**Implementado**
Serviço de Entregas -> MongoDB Atlas	Driver MongoDB sobre TLS	Persistência das entregas.	**Implementado**
Serviço de Entregas -> Barramento de Eventos	AMQP/TLS (Azure Service Bus)	Publicação de eventos de domínio.	**Implementado**
Barramento -> Serviço de Alertas	ServiceBusQueueTrigger	Consumo de eventos para alertas.	**Implementado**
GPS/IoT -> Telemetria	MQTT (Azure IoT Hub)	Eventos de posição, timestamp e identificador do veículo.	**Planejado**

Interface	Canal	Finalidade	Situação
CloudLog -> Notificações	HTTPS/API	Solicitação de push, e-mail ou SMS.	**Planejado**
CloudLog -> Provedor de Identidade	OpenID Connect / OAuth 2.0	Login com MFA e emissão de JWT.	**Planejado**
Shell Web -> Microfrontends	Module Federation (remoteEntry.js)	Composição do frontend em tempo de execução.	**Planejado**

4. Estratégia de Solução

A estratégia de solução prioriza uma arquitetura cloud modular, separando interfaces de usuário, processamento de telemetria, regras de negócio, persistência e funcionalidades assíncronas. Na versão completa, essa modularidade é concretizada com microsserviços serverless em Azure Functions, cada um dono do seu banco, expostos por um API Gateway e por BFFs específicos de cada canal, integrados por eventos e consumidos por microfrontends. A escolha responde aos principais drivers de qualidade: disponibilidade, segurança, desempenho, escalabilidade e facilidade de evolução.

Princípio / decisão	Racional
Separação Web e Mobile	Permite adaptar experiência e ciclo de evolução aos diferentes perfis de uso.
Microsserviços por capacidade de negócio (substitui a API central da v0.1)	Cada capacidade (entregas, telemetria, comprovantes, alertas, indicadores) evolui e escala de forma independente.
Database per Service	Cada serviço é dono exclusivo dos seus dados; integração apenas por API ou evento.
API Gateway + BFF por canal	Uma porta única aplica segurança e limites; os BFFs adaptam as respostas para Web e Mobile.
Microfrontends no cliente Web	Telas por domínio publicadas de forma independente dentro de um Shell.
Telemetria como serviço separado	Isola alto volume de eventos e permite escalar ingestão independentemente.
Processamento assíncrono por eventos (EDA)	Reduz acoplamento e evita bloquear operações de negócio por dependências externas.
Serverless (Azure Functions, Consumption)	Escala automática, custo zero ocioso e operação simplificada.
Clean Architecture + Vertical Slice dentro de cada serviço	Domínio testável sem infraestrutura e mudanças localizadas por funcionalidade.
Testes unitários de arquitetura no CI	As regras de dependência deixam de ser só documentação e passam a ser verificadas a cada push.
Observabilidade transversal	Logs, métricas e traces sustentam operação, diagnóstico e evolução.

4.1 Software Architecture Canvas

Software Architecture Canvas - CloudLog

Problema e contexto	Stakeholders	Requisitos-chave
Baixa visibilidade da frota; falhas na gestão de rotas; atraso na comunicação entre central e motoristas; comprovantes em papel.	Gerente de Logística (Carlos); Motorista (Marcos); Atendente de SAC (Fernanda); equipe de desenvolvimento; operação de TI; gestão da empresa.	RF-01 autenticação/RBAC; RF-02 rastreamento; RF-03 rotas/entregas (implementado); RF-04 comprovante; RF-05 alertas (atraso implementado); RF-06 indicadores.
Atributos de qualidade	**Decisões arquiteturais**	**Integrações e dados**
Disponibilidade; segurança; desempenho; usabilidade mobile; escalabilidade; manutenibilidade e testabilidade.	Serverless (Azure Functions); microsserviços com Database per Service; API Gateway (APIM); BFF Web e Mobile; microfrontends com Module Federation; EDA com Azure Service Bus; Clean Architecture + Vertical Slice; testes de arquitetura no CI.	GPS/IoT via IoT Hub; Provedor de Identidade (Entra External ID); Serviço de Notificações; MongoDB Atlas (entregas); Cosmos DB (telemetria); Blob (comprovantes); Table Storage (alertas); Azure SQL (indicadores).
Restrições	**Riscos**	**Operação e evolução**
Azure for Students; Static Web Apps só aceita gatilho HTTP; MongoDB Atlas M0 pausa sem uso; conectividade móvel variável; MFA/RBAC; proteção de dados; equipe de três alunos.	Perda de evento sem outbox; rotas ainda sem autenticação; cold start; complexidade operacional de microsserviços; custos fora da conta estudante; consistência eventual.	GitHub Actions (testes + deploy); Application Insights; diagramas como código; extração gradual dos serviços planejados usando o Serviço de Entregas como modelo.

Figura 2 - Software Architecture Canvas do CloudLog (versão 1.0).

O Canvas sintetiza os principais drivers arquiteturais e funciona como ponte entre requisitos, decisões, riscos e a decomposição apresentada nas próximas seções.

4.2 Estilos arquiteturais adotados

Estilo	Como é aplicado no CloudLog	Onde no documento	Situação
Serverless	Todo o backend roda em Azure Functions no plano de consumo; banco, mensageria e hospedagem são serviços gerenciados.	5.1, 7	**Implementado**
Microfrontend	Shell React (host) carrega MFE Entregas, MFE Operação e MFE Rastreamento via Module Federation, cada um com deploy próprio.	5.1	**Parcial** (telas numa SPA)
Backend for Frontend (BFF)	BFF Web agrega entregas, posições e indicadores para gerente e SAC; BFF Mobile entrega payload enxuto ao motorista.	5.1, 6.3	**Planejado**
Microservices + Database per Service	Serviços de Entregas, Telemetria, Comprovantes, Alertas e Indicadores, cada um com banco exclusivo.	5.1, 5.5	**Parcial** (Entregas e Alertas)
API Gateway	Azure API Management valida JWT e perfil, aplica rate limit e roteia para os BFFs. Hoje o roteamento /api é do Static Web Apps.	5.1, 7	**Parcial**
Arquitetura Orientada a Eventos (EDA)	CRUD publica DeliveryCreated, DeliveryUpdated e DeliveryDeleted no Service Bus; o Serviço de Alertas consome e gera alerta de atraso.	6.1, 6.4	**Implementado**
Clean Architecture + Vertical Slice	Backend organizado em features/<funcionalidade> com domínio puro no centro, adaptadores na borda e Composition Root.	5.2, 5.3, 5.6	**Implementado**
Testes unitários de arquitetura	13 testes com node:test leem as dependências de api/src e falham se as regras forem violadas; executados no GitHub Actions.	8.1	**Implementado**

5. Visão de Blocos de Construção

A visão de blocos de construção apresenta a decomposição estática do CloudLog. Primeiro é mostrado o C4 Nível 2, com os contêineres executáveis/implantáveis; em seguida o C4 Nível 3, com os componentes internos do Serviço de Entregas, que é o serviço implementado. O detalhamento de Nível 4 utiliza UML conforme orientação da disciplina.

5.1 C4 Model - Nível 2: Contêineres

O diagrama da Figura 3 reúne os estilos adotados: microfrontends no cliente web, API Gateway e BFFs na borda, microsserviços serverless com banco exclusivo e um barramento de eventos. Elementos em azul escuro estão implementados; em azul claro fazem parte da arquitetura-alvo.

Contêiner	Responsabilidade	Tipo/tecnologia arquitetural	Situação
Shell Web	Login, layout, menu e carregamento dos microfrontends.	React + Vite (host Module Federation)	**Parcial**
MFE Entregas	Pesquisa, inserção, alteração e exclusão de entregas.	React (remote)	**Parcial**
MFE Operação	Indicadores, telemetria resumida e alertas.	React (remote)	**Parcial**
MFE Rastreamento	Mapa da frota em tempo real.	React (remote)	**Planejado**
App do Motorista	Interface operacional do motorista e comprovação digital.	React Native	**Planejado**
API Gateway	Autenticação na borda, rate limit, CORS e roteamento.	Azure API Management	**Planejado**
BFF Web / BFF Mobile	Agregação e adaptação das respostas por canal.	Azure Functions	**Planejado**
Serviço de Entregas	CRUD de entregas e publicação de eventos.	Azure Functions + MongoDB Atlas	**Implementado**
Serviço de Alertas	Consome eventos e gera alertas de atraso.	Azure Functions (ServiceBusQueueTrigger)	**Implementado**
Serviço de Telemetria	Ingestão/normalização de eventos GPS/IoT.	Azure Functions + Cosmos DB	**Planejado**
Serviço de Comprovantes	Fotos, assinaturas e URLs SAS.	Azure Functions + Blob Storage	**Planejado**
Serviço de Indicadores	Projeções de leitura para dashboards.	Azure Functions + Azure SQL	**Planejado**
Barramento de Eventos	Transporte dos eventos de domínio.	Azure Service Bus	**Implementado**

Microfrontends

O Shell é o único ponto acessado pelo usuário: faz o login e carrega cada microfrontend sob demanda. Cada MFE é publicado de forma independente e conversa apenas com o BFF Web. Hoje as telas Operação e Entregas existem em uma única SPA React; a separação em remotes está registrada como dívida técnica (Seção 11).

Microfrontend	Rota no Shell	Módulo exposto	Consome
Shell (host)	/	—	remoteEntry.js dos MFEs
MFE Entregas	/entregas	./DeliveriesPage	BFF Web /web/deliveries
MFE Operação	/operacao	./OperationsDashboard	BFF Web /web/dashboard
MFE Rastreamento	/rastreamento	./FleetMap	BFF Web /web/positions

API Gateway e Backend for Frontend

Rota pública (API Gateway)	Destino	Políticas na borda	Papel do BFF
GET /web/dashboard	BFF Web	JWT válido; perfil Gerente ou SAC; rate limit	Agrega Entregas, Telemetria e Indicadores (Figura 15)
GET/POST/PUT/DELETE /web/deliveries	BFF Web -> Serviço de Entregas	JWT; escrita só para Gerente	Repasse com adaptação de payload
GET /mobile/today	BFF Mobile	JWT; perfil Motorista	Somente as entregas do motorista, campos mínimos
POST /mobile/proofs	BFF Mobile -> Serviço de Comprovantes	JWT; limite de upload	Gera URLs SAS para envio direto ao Blob

Microservices e Database per Service

Microsserviço	Capacidade	Banco exclusivo	Publica	Consome
Entregas (**Implementado**)	Rotas e entregas (RF-03)	MongoDB Atlas cloudlog.deliveries	DeliveryCreated, DeliveryUpdated, DeliveryDeleted	ProofSubmitted (alvo)
Alertas (**Implementado**)	Alertas (RF-05)	Table Storage	AlertRaised (alvo)	DeliveryCreated, DeliveryUpdated, RouteDeviationDetected
Telemetria (**Planejado**)	Rastreamento (RF-02)	Cosmos DB	PositionReceived, RouteDeviationDetected	Mensagens do IoT Hub
Comprovantes (**Planejado**)	Comprovação (RF-04)	Blob Storage + metadados	ProofSubmitted	—
Indicadores (**Planejado**)	Dashboards (RF-06)	Azure SQL	—	Todos os eventos

5.1 C4 Model - Nível 2: Contêineres

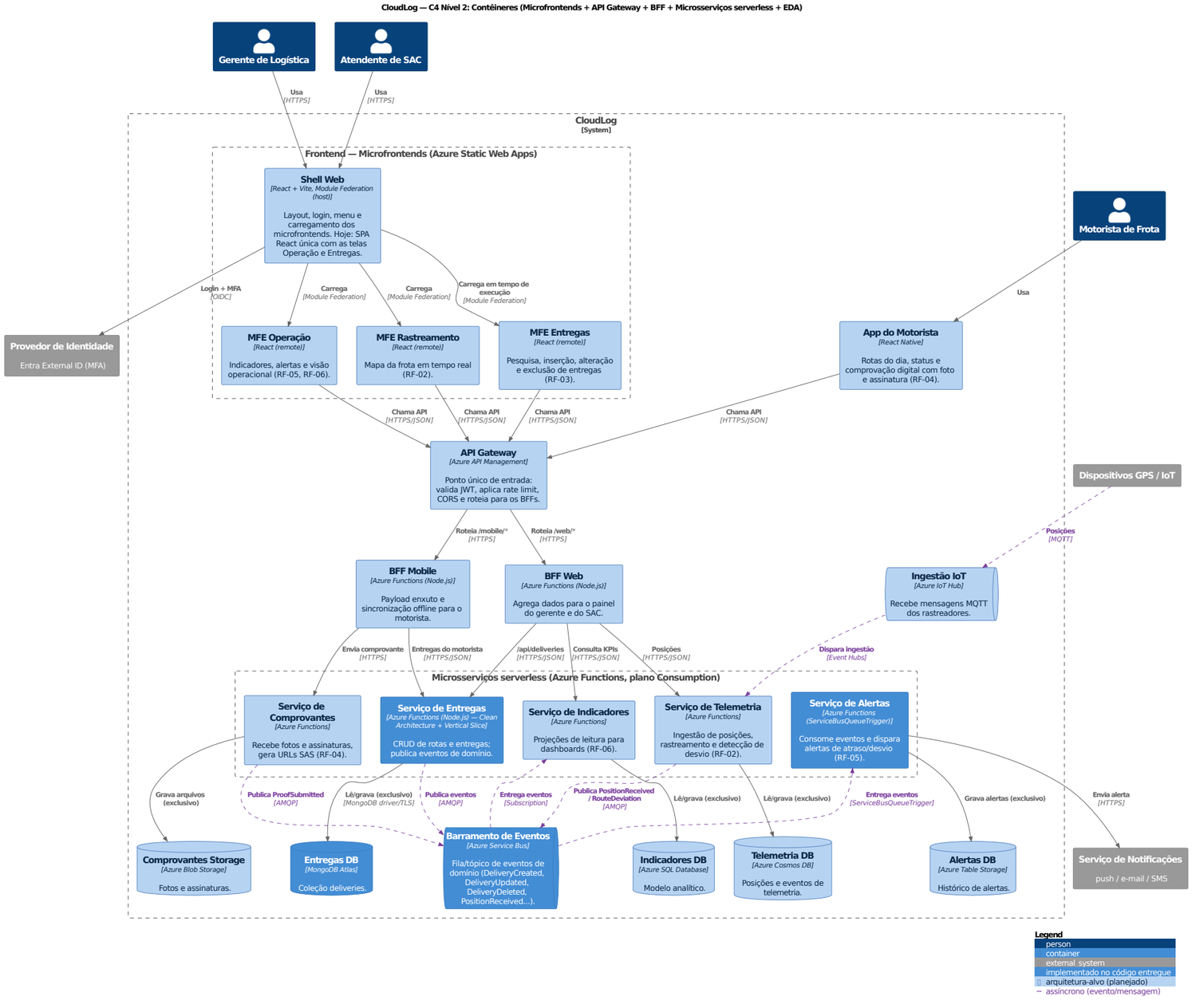


Figura 3 - C4 Nível 2: contêineres do CloudLog.

5.2 C4 Model - Nível 3: Componentes do Serviço de Entregas

Na v0.1 o Nível 3 detalhava uma API/Backend única. Na v1.0, o contêiner detalhado é o Serviço de Entregas, e os componentes correspondem exatamente às pastas do código em `api/src`. O consumidor `processDeliveryEvent` está no mesmo repositório, mas é implantado como outro contêiner, o Serviço de Alertas.

Componente	Rastreabilidade	Responsabilidade
HTTP Functions (features/*/*.function.js)	RF-03	Registram as rotas createDelivery, searchDeliveries, updateDelivery, deleteDelivery e helloCloudLog.
Handlers HTTP (features/*/*.handler.js + shared/http)	RF-03	Leem a requisição, chamam o caso de uso e traduzem erros em status HTTP.
Casos de Uso (features/*/*UseCase.js)	RF-03, RF-05	Orquestram a operação e publicam eventos depois de persistir.
Domínio (domain/)	RF-03	Entidade Delivery, DeliveryStatus, DeliveryEvents e erros.
Portas (domain/ports)	Transversal	Contratos exigidos pelos casos de uso (DIP/ISP).
MongoDeliveryRepository	RF-03	Implementa as portas de persistência sobre o MongoDB Atlas.
ServiceBusDeliveryEventPublisher	RF-05	Implementa a porta de publicação de eventos.
Composition Root (bootstrap/container.js)	Transversal	Instancia adaptadores e injeta nos casos de uso.

5.2 C4 Model - Nível 3: Componentes do Serviço de Entregas

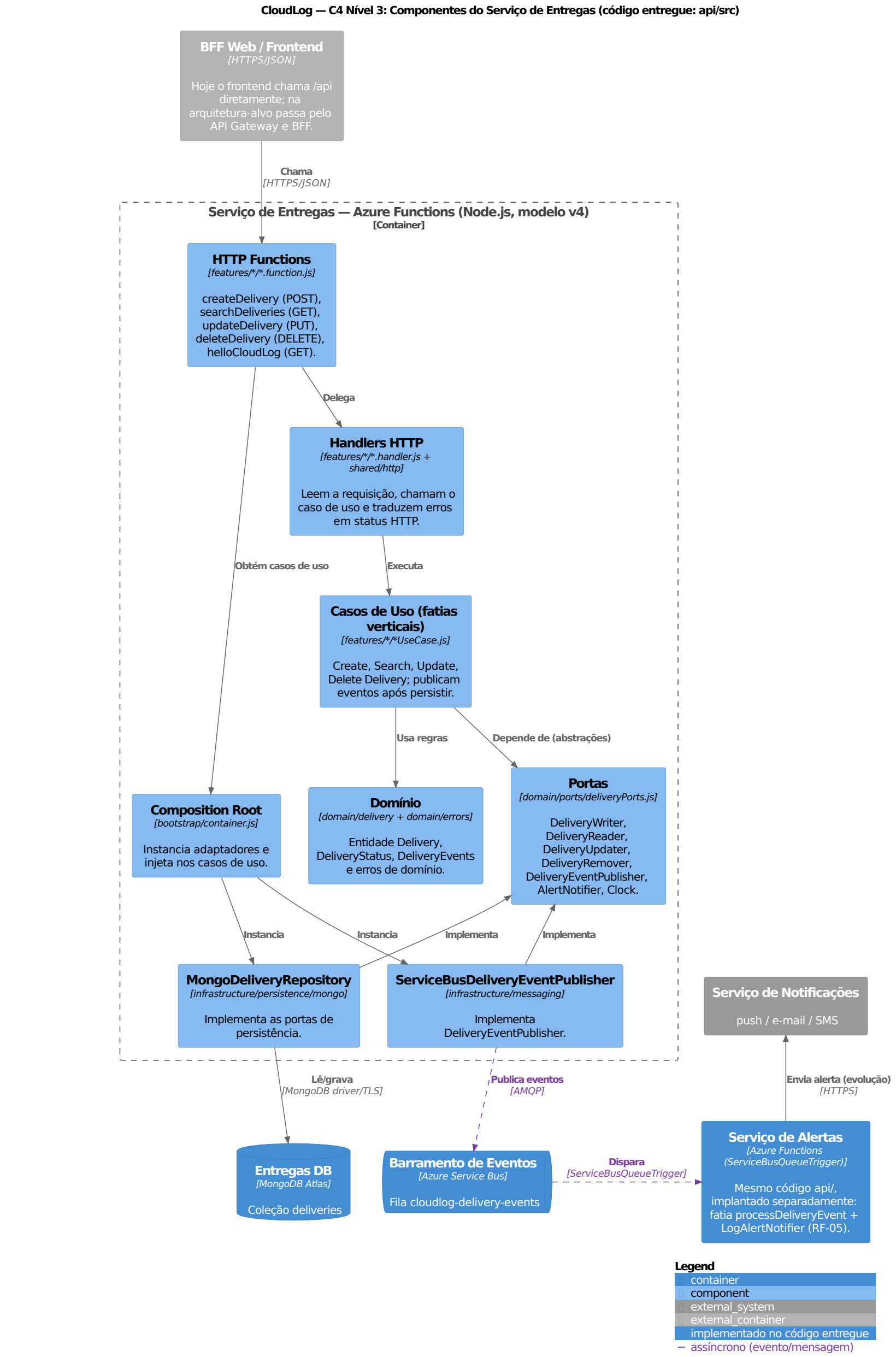


Figura 4 - C4 Nível 3: componentes internos do Serviço de Entregas.

5.3 C4 Nível 4 / UML - Diagrama de Classes

O Nível 4 é apresentado em três diagramas de classes. O primeiro mantém o modelo de domínio conceitual da v0.1, agora com o perfil Atendente de SAC. Os dois seguintes mostram as classes reais do código do Serviço de Entregas.

C4 Nível 4 / UML — Modelo de Domínio do CloudLog (visão conceitual)

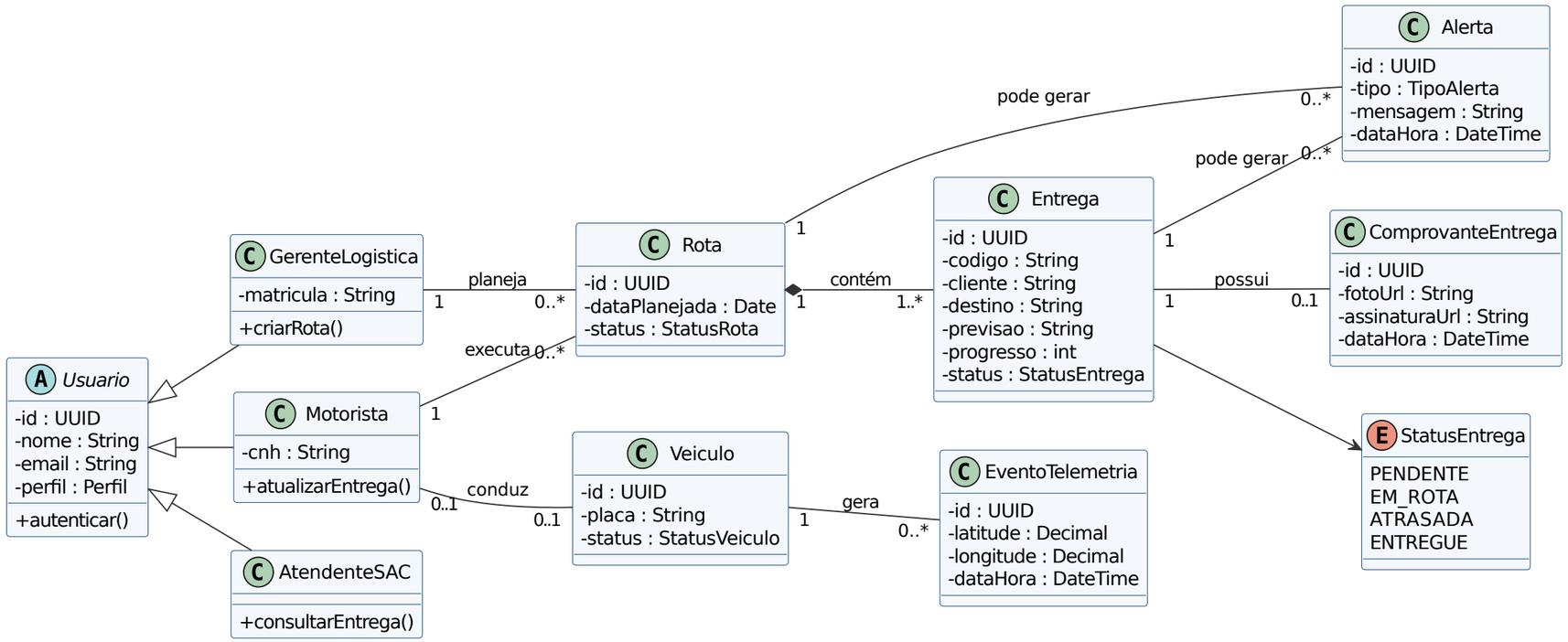


Figura 5 - UML de Classes: modelo de domínio conceitual do CloudLog.

Usuário é generalizado em GerenteLogística, Motorista e AtendenteSAC; uma Rota contém uma ou mais Entregas; a Entrega pode possuir um ComprovanteEntrega; Veículos geram EventosTelemetria; rotas e entregas podem originar Alertas.

5.3 C4 Nível 4 / UML - Diagrama de Classes (código 1/2)

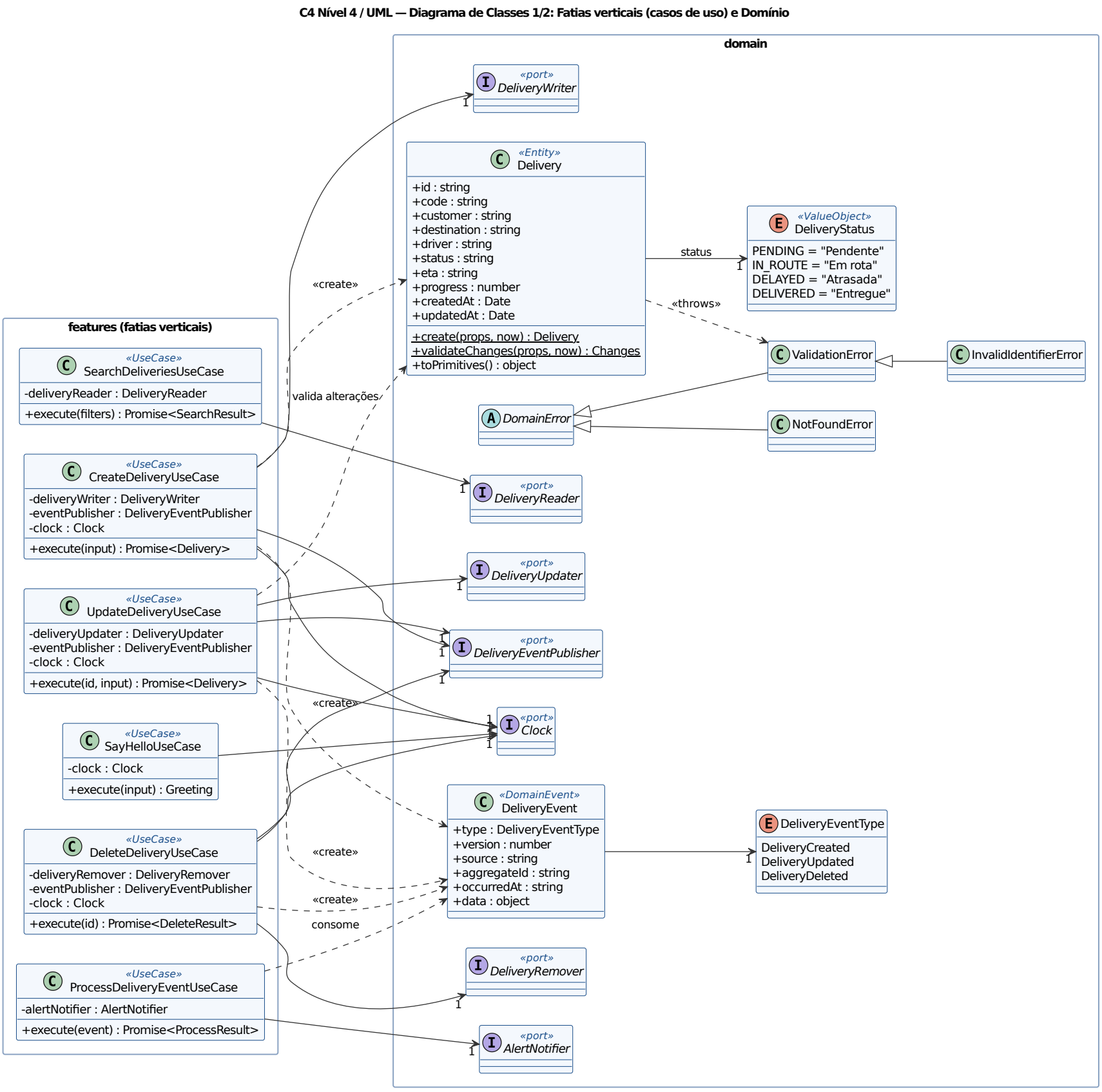


Figura 6 - UML de Classes (código): casos de uso das fatias verticais e domínio.

5.3 C4 Nível 4 / UML - Diagrama de Classes (código 2/2)

C4 Nível 4 / UML — Diagrama de Classes 2/2: Portas e Adaptadores (DIP / LSP / ISP)

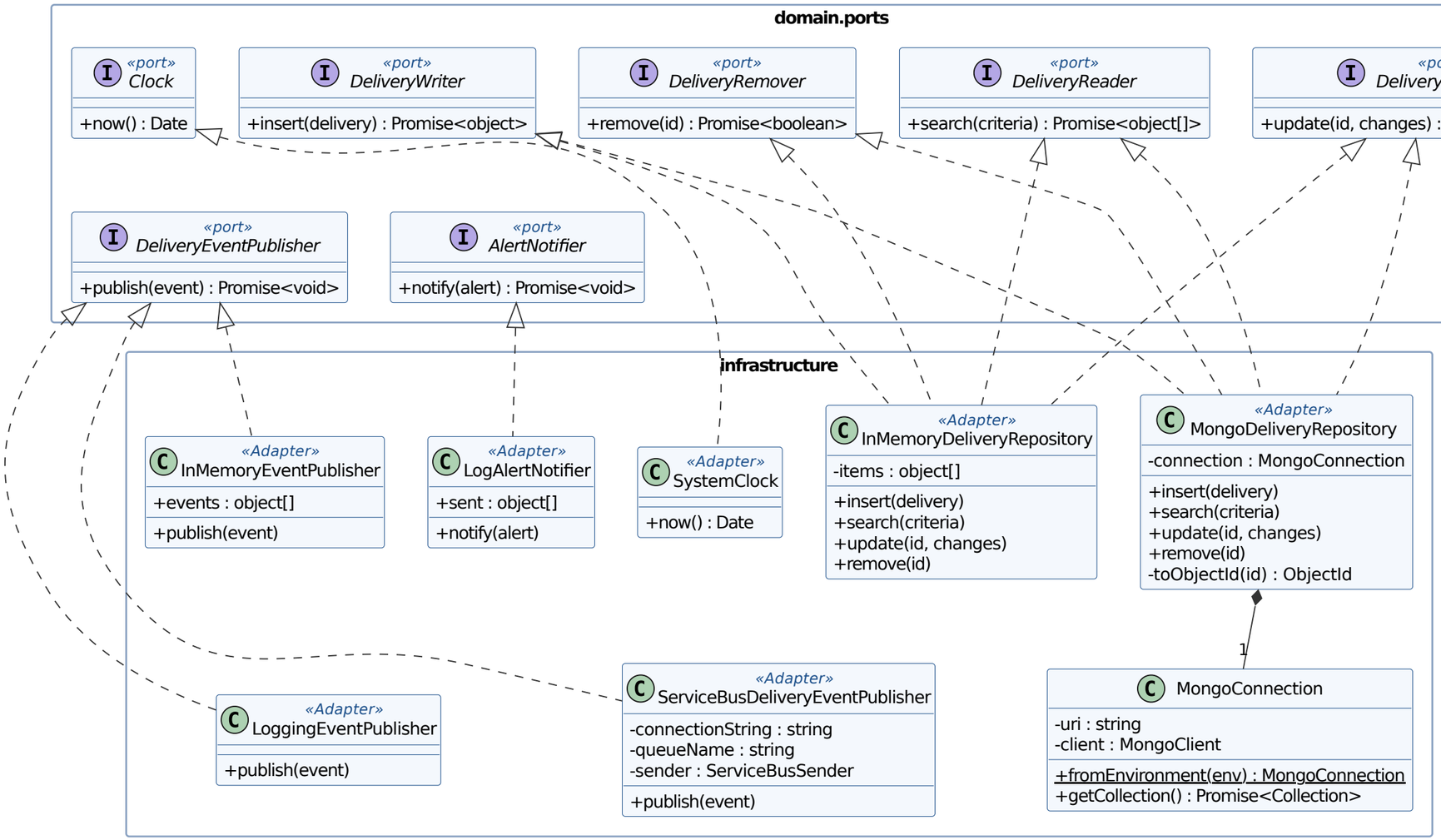


Figura 7 - UML de Classes (código): portas do domínio e adaptadores que as realizam.

Os casos de uso dependem apenas de portas (interfaces). Os adaptadores realizam essas portas e são intercambiáveis: MongoDB ou memória para persistência; Service Bus, log ou memória para eventos. Essa substituição é usada nos testes unitários.

5.4 C4 Nível 4 / UML - Diagrama de Componentes

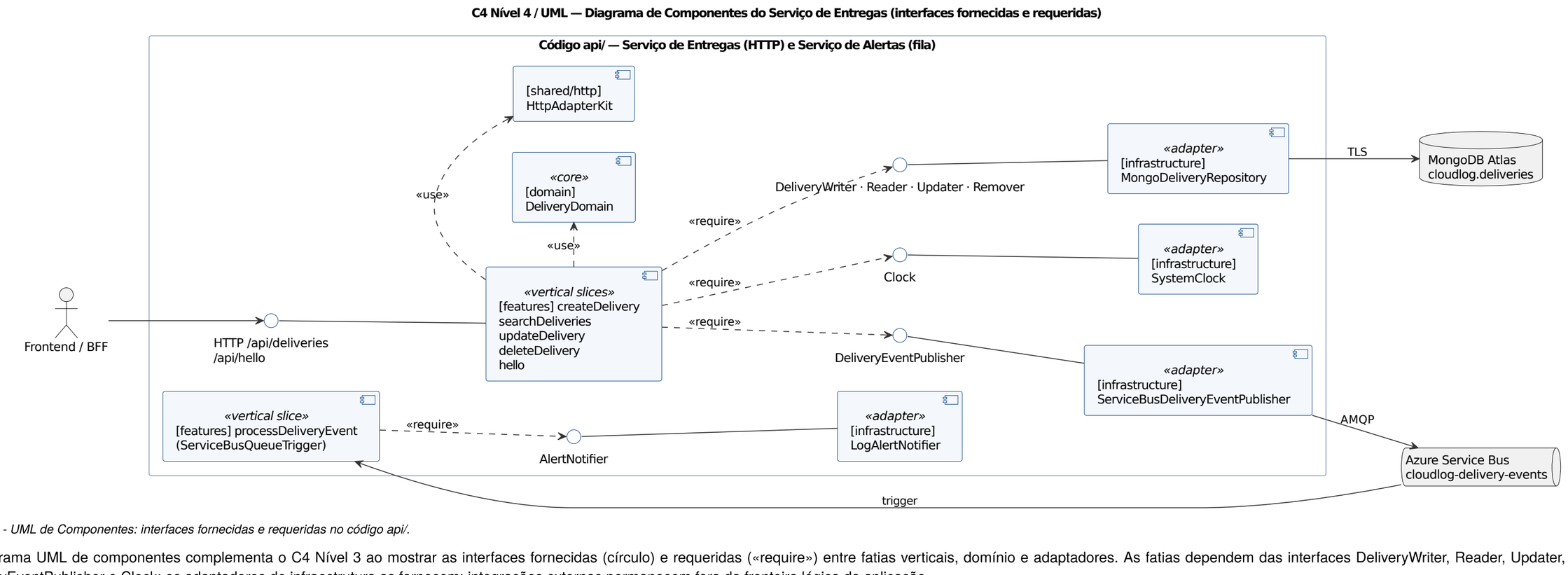


Figura 8 - UML de Componentes: interfaces fornecidas e requeridas no código api/.

O diagrama UML de componentes complementa o C4 Nível 3 ao mostrar as interfaces fornecidas (círculo) e requeridas («require») entre fatias verticais, domínio e adaptadores. As fatias dependem das interfaces DeliveryWriter, Reader, Updater, Remover, DeliveryEventPublisher e Clock; os adaptadores de infraestrutura as fornecem; integrações externas permanecem fora da fronteira lógica da aplicação.

5.5 Diagrama de Entidades e Relacionamentos (DER)

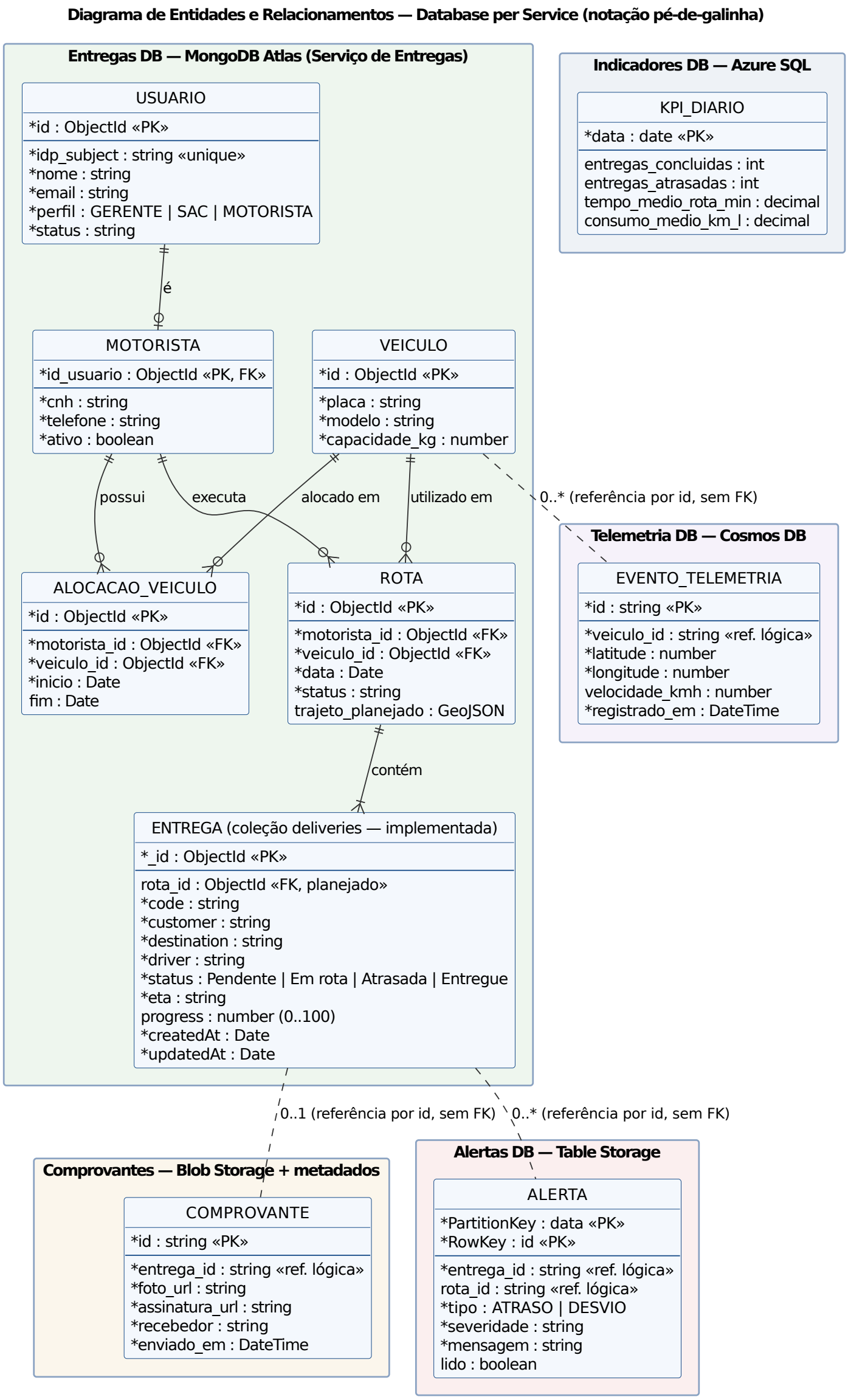


Figura 9 - DER organizado por banco de serviço (Database per Service).

O DER separa o modelo de persistência do modelo de classes e, na v1.0, está agrupado por banco de serviço. Chaves estrangeiras existem apenas dentro do mesmo banco (Entregas DB: usuário, motorista, veículo, alocação, rota e entrega). Entre serviços há somente referência lógica por identificador, sem integridade referencial no banco, mantida por eventos. A entidade ALOCACAO_VEICULO registra o histórico de associação entre motorista e veículo. A entidade ENTREGA corresponde à coleção `deliveries`, já em uso no MongoDB Atlas; as demais pertencem à arquitetura-alvo.

5.6 Clean Architecture e Vertical Slice

As duas abordagens se complementam no código entregue: **Vertical Slice** define como o código é agrupado (uma pasta por funcionalidade, de ponta a ponta) e **Clean Architecture** define a direção permitida das dependências (sempre para o domínio).

```
api/src/
├── index.js                        registra as fatias
├── bootstrap/container.js          Composition Root (injeção de dependência)
├── domain/                         núcleo sem dependências externas
│   ├── delivery/Delivery.js · DeliveryStatus.js · DeliveryEvents.js
│   ├── errors.js
│   └── ports/deliveryPorts.js      Writer, Reader, Updater, Remover, EventPublisher, AlertNotifier, Clock
├── features/                       FATIAS VERTICAIS (function + handler + caso de uso)
│   ├── createDelivery/  searchDeliveries/  updateDelivery/  deleteDelivery/  hello/
│   └── processDeliveryEvent/       consumidor ServiceBusQueueTrigger (Serviço de Alertas)
├── infrastructure/                 persistence/mongo · persistence/memory · messaging · notifications · time
└── shared/http/                    httpResponse · requestReader · errorMapper
```

Princípio SOLID	Evidência no código
S - Responsabilidade Única	Validação na entidade Delivery; orquestração no caso de uso; tradução HTTP no handler; persistência no repositório.
O - Aberto/Fechado	Nova funcionalidade é uma nova pasta em features/; novo erro é uma linha em errorStatusTable; novo destino de eventos é um novo adaptador.
L - Substituição de Liskov	Repositórios Mongo e memória e os três publicadores de eventos são intercambiáveis; assertPort verifica o contrato.
I - Segregação de Interface	Portas pequenas: cada caso de uso recebe apenas a porta que usa.
D - Inversão de Dependência	Casos de uso dependem de portas; somente o Composition Root instancia classes concretas.

5.6 Clean Architecture e Vertical Slice

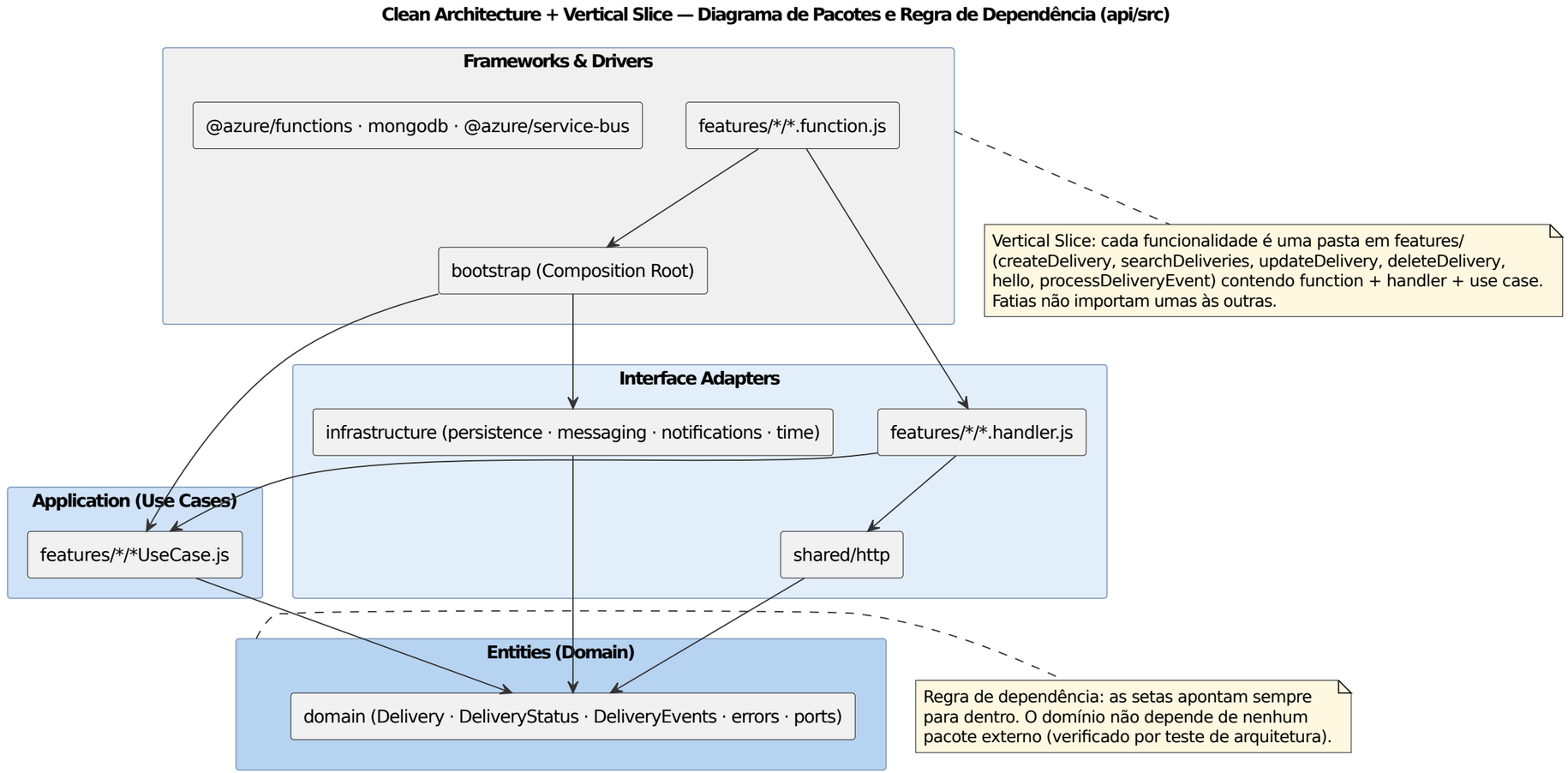


Figura 10 - Pacotes da Clean Architecture aplicados às fatias verticais (regra de dependência).

6. Visão de Execução

A visão de execução descreve o comportamento dinâmico. Na v1.0 há cinco diagramas de sequência UML: dois cenários já implementados no código (inserção de entrega com evento e alerta de atraso) e três cenários da arquitetura-alvo (conclusão de entrega com comprovante, rastreamento com desvio e consulta do dashboard via BFF).

6.1 C4 Nível 4 / UML - Diagramas de Sequência

Inserir entrega (implementado). O handler lê o JSON e chama o caso de uso. A entidade valida os dados; se inválidos, a API responde 400 sem acessar o banco. Se válidos, o repositório grava no MongoDB, o caso de uso publica DeliveryCreated e a API responde 201. Nenhum evento é publicado quando a operação falha.

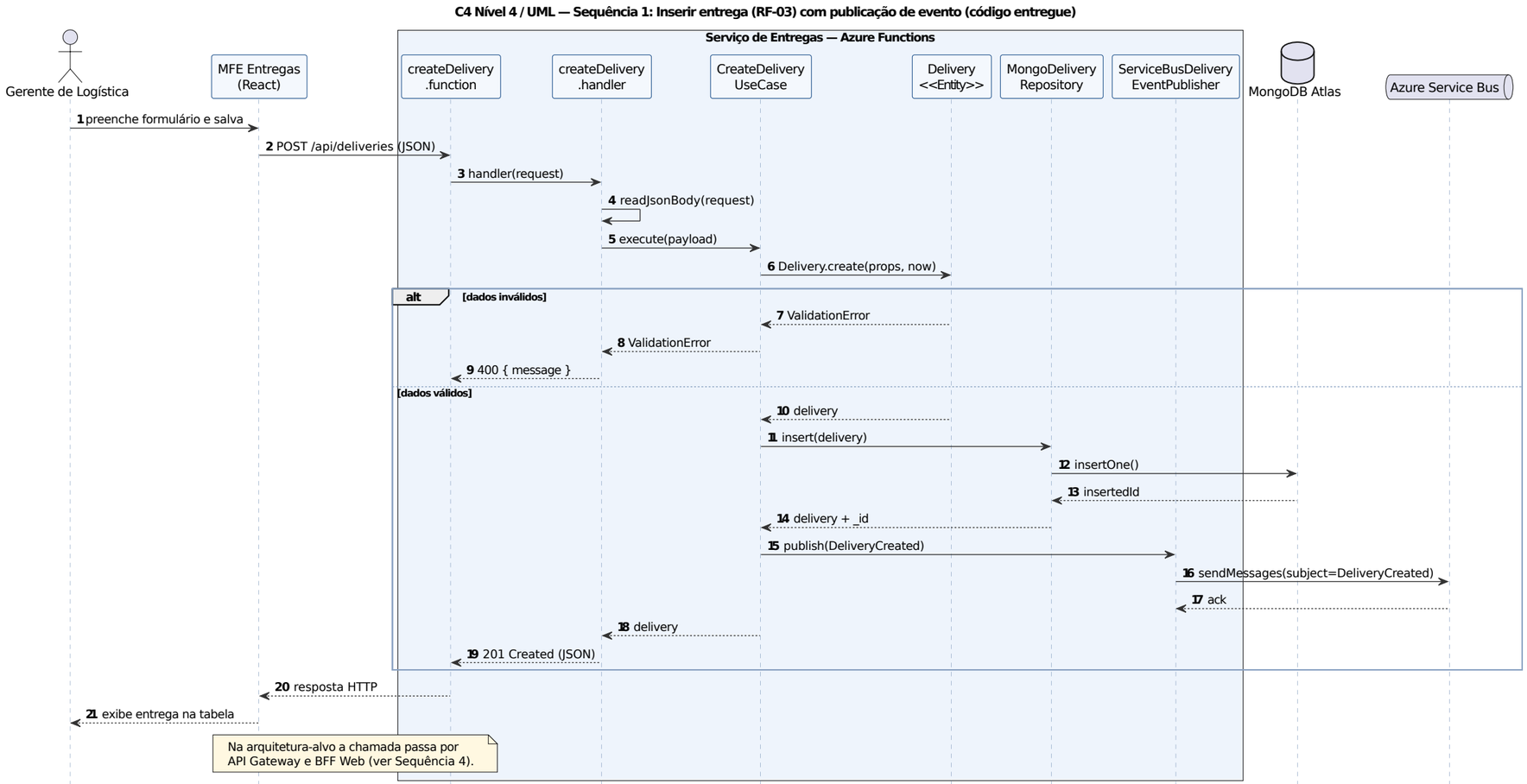


Figura 11 - UML de Sequência: inserção de entrega com publicação de DeliveryCreated.

6.1 C4 Nível 4 / UML - Sequência 2: Alerta de atraso

Alerta de atraso por eventos (implementado). Quando a entrega passa a Atrasada, a API responde assim que grava e publica DeliveryUpdated. O alerta acontece depois, de forma assíncrona: o Service Bus aciona o consumidor processDeliveryEvent, que notifica a central. Se o consumidor estiver indisponível, a mensagem aguarda na fila e a API não é afetada.

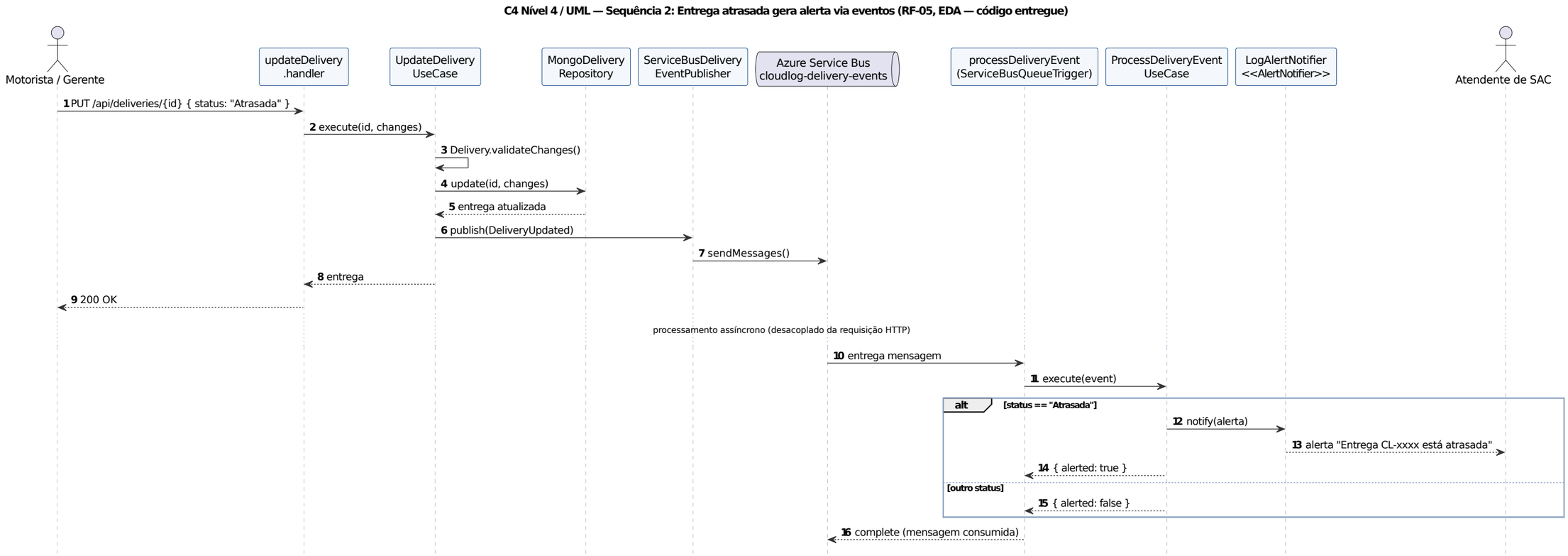


Figura 12 - UML de Sequência: DeliveryUpdated -> Service Bus -> processDeliveryEvent -> alerta.

6.1 C4 Nível 4 / UML - Sequência 3: Comprovante digital

Conclusão de entrega com comprovante digital (arquitetura-alvo). Cenário principal da v0.1, agora distribuído: o app envia fotos e assinatura direto ao Blob Storage usando URLs SAS obtidas pelo BFF Mobile, o Serviço de Comprovantes publica ProofSubmitted e o Serviço de Entregas conclui a entrega ao consumir o evento.

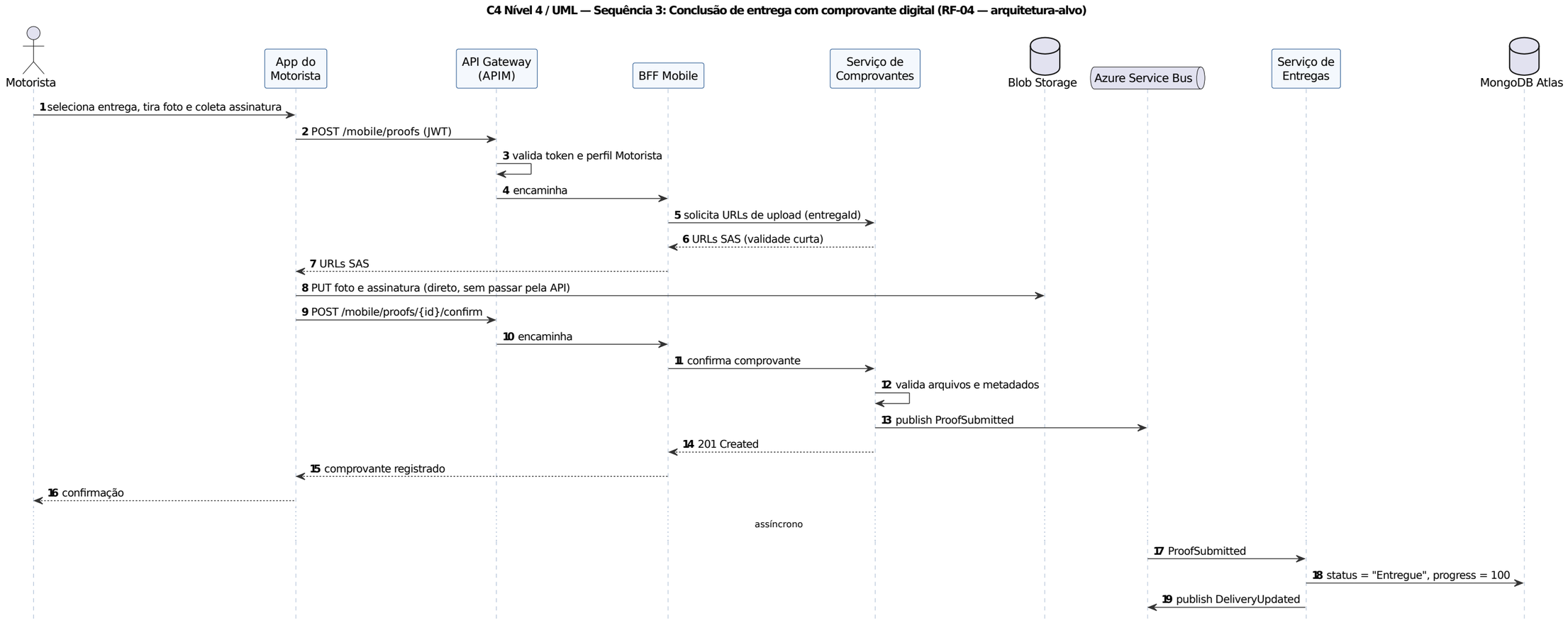


Figura 13 - UML de Sequência: conclusão de entrega e registro do comprovante digital.

6.2 Cenário - rastreamento e alerta de desvio

1. O dispositivo GPS/IoT publica um evento de localização contendo veículo, coordenadas e timestamp no IoT Hub (MQTT).
2. O Serviço de Telemetria valida e normaliza o evento e grava a posição no seu banco (Cosmos DB).
3. A posição é comparada com a rota planejada e o prazo.
4. Quando o veículo sai do corredor da rota, é publicado o evento RouteDeviationDetected.
5. O Serviço de Alertas consome o evento e solicita ao provedor externo o envio da notificação; a central passa a visualizar a ocorrência.
6. Em paralelo, PositionReceived atualiza as projeções do Serviço de Indicadores.

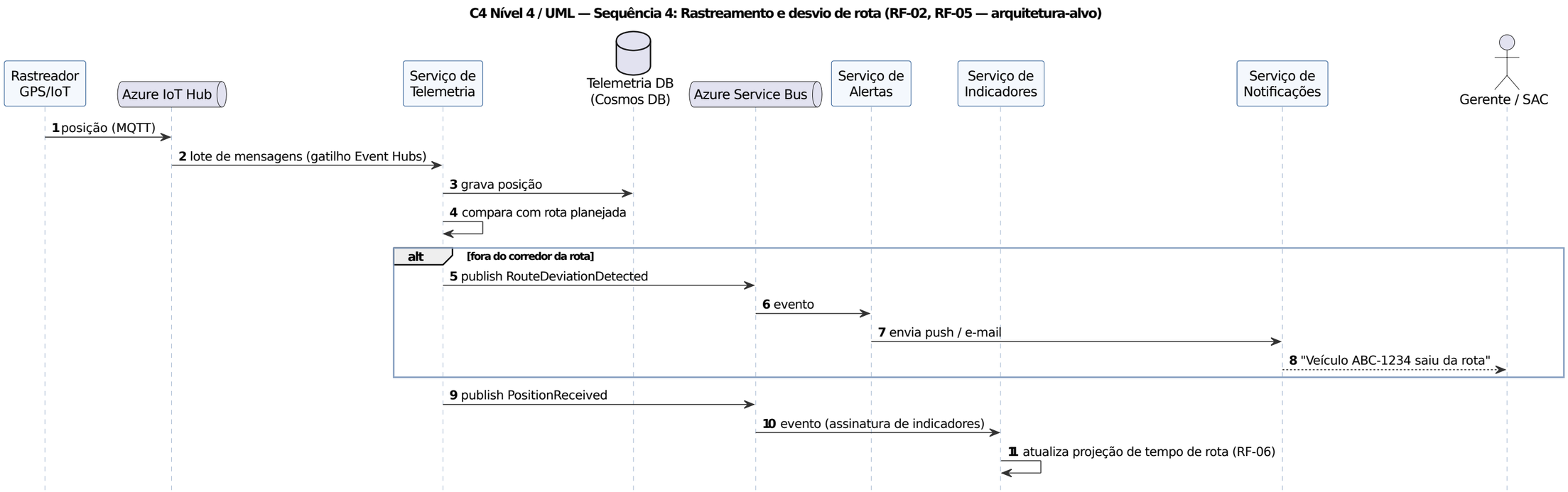


Figura 14 - UML de Sequência: ingestão de telemetria e alerta de desvio.

6.3 Cenário - consulta do dashboard

O gerente acessa o MFE Operação, que faz uma única chamada ao API Gateway. O gateway valida o token e o perfil e encaminha ao BFF Web, que consulta Entregas, Telemetria e Indicadores em paralelo e devolve uma resposta pronta para a tela, com métricas de tempo de rota, consumo e eficiência, respeitando as permissões do perfil autenticado.

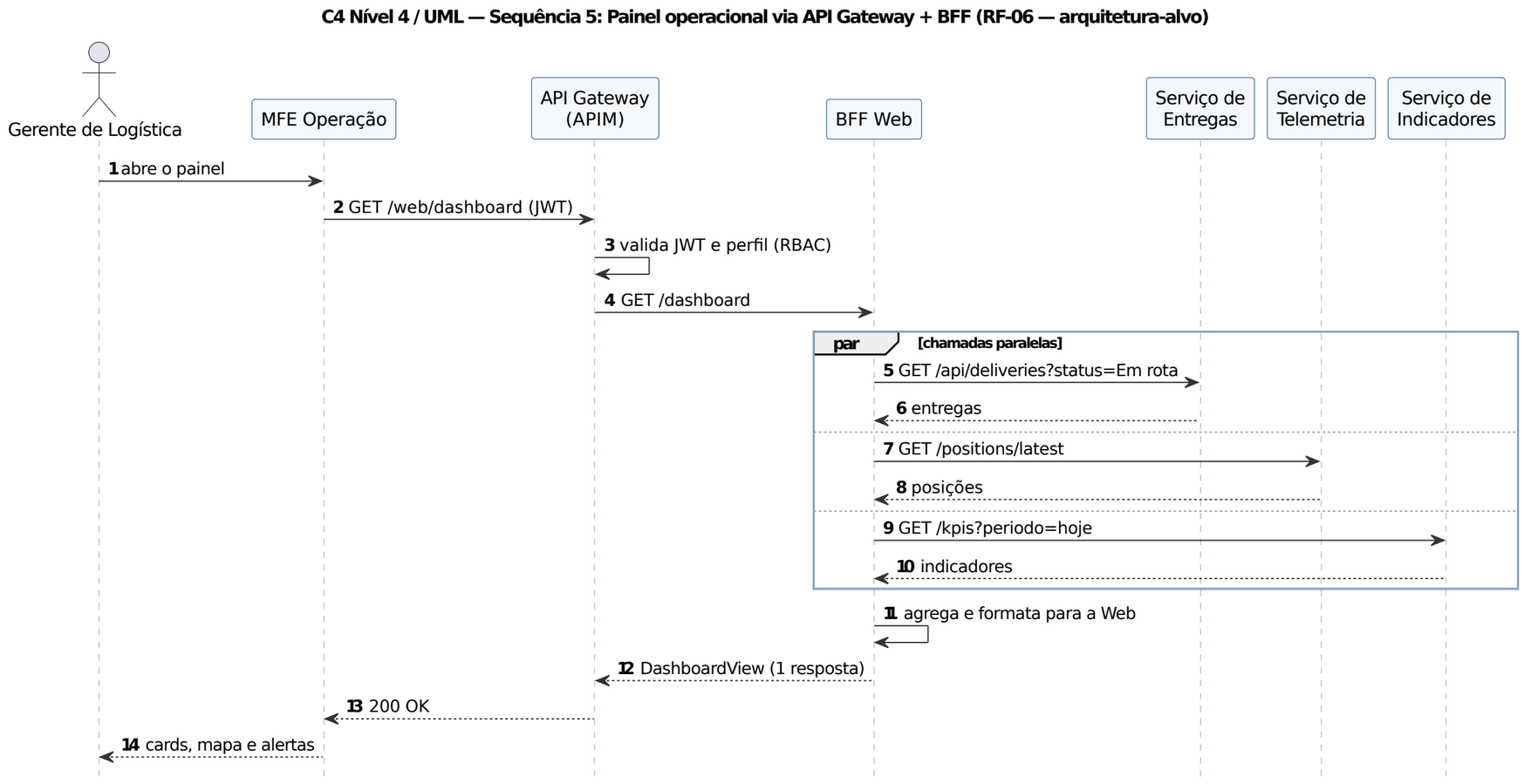


Figura 15 - UML de Sequência: painel operacional via API Gateway e BFF Web.

6.4 Catálogo de eventos

Evento	Produtor	Consumidores	Quando	Situação
DeliveryCreated	Serviço de Entregas	Alertas, Indicadores	Após inserir uma entrega	**Implementado**
DeliveryUpdated	Serviço de Entregas	Alertas, Indicadores	Após alterar; inclui changedFields	**Implementado**
DeliveryDeleted	Serviço de Entregas	Indicadores	Após excluir	**Implementado**
PositionReceived	Telemetria	Indicadores	A cada posição recebida	**Planejado**
RouteDeviationDetected	Telemetria	Alertas	Veículo fora da rota	**Planejado**
ProofSubmitted	Comprovantes	Entregas, Indicadores	Após upload de foto e assinatura	**Planejado**


```
{ "type": "DeliveryUpdated", "version": 1, "source": "cloudlog/delivery-service",
  "aggregateId": "6720f1c2a9e4b1d3c0f5a911", "occurredAt": "2026-10-14T13:05:00.000Z",
  "data": { "code": "CL-2050", "status": "Atrasada", "driver": "Marcos Silva", "changedFields": ["status", "updatedAt"] } }
```

7. Visão de Implantação

Na v0.1 a implantação era propositalmente provider-neutral. Na v1.0 ela utiliza Microsoft Azure com serviços serverless e gerenciados, escolhidos pelo custo gratuito ou por consumo na assinatura de estudante, e o MongoDB Atlas para o Serviço de Entregas.

Camada	Elemento	Responsabilidade	Situação
Borda/entrada	Azure API Management (Consumption)	Autenticação, limites e roteamento para os BFFs.	**Planejado**
Frontend	Azure Static Web Apps	Entrega da aplicação web (hoje SPA; alvo: Shell + MFEs).	**Implementado**
Backend	Funções gerenciadas do Static Web Apps	Serviço de Entregas (/api/deliveries, /api/hello).	**Implementado**
BFFs	Function Apps (Consumption)	BFF Web e BFF Mobile.	**Planejado**
Telemetria	Azure IoT Hub + Function App + Cosmos DB	Recepção e normalização de alto volume de eventos.	**Planejado**
Assíncrono	Azure Service Bus (Basic) + Function App consumidora	Desacopla alertas e integrações externas.	**Parcial** (configurável)
Dados	MongoDB Atlas (M0) e bancos por serviço	Persistência por serviço, backups e recuperação.	**Implementado** (Entregas)
Arquivos	Azure Blob Storage	Evidências digitais e comprovantes.	**Planejado**
Operação	Application Insights + GitHub Actions	Logs, métricas, testes e deploy contínuo.	**Parcial**

Configuração	Uso
MONGODB_ATLAS_URI, MONGODB_ATLAS_DATABASE, MONGODB_ATLAS_COLLECTION	Conexão do Serviço de Entregas com o MongoDB Atlas.
SERVICEBUS_CONNECTION, SERVICEBUS_QUEUE	Com a conexão definida, eventos vão para o Service Bus; sem ela, apenas para o log.
SERVICEBUS_CONSUMER_ENABLED=true	Registra o gatilho de fila na Function App consumidora (o Static Web Apps só aceita HTTP).
DELIVERY_REPOSITORY=memory, EVENT_PUBLISHER=memory	Execução local e testes sem banco e sem Service Bus.

Pipeline: todo push ou Pull Request que altera `api/` executa os testes unitários e de arquitetura no GitHub Actions; o merge na `main` dispara o deploy do Static Web Apps (frontend e API).

7. Visão de Implantação

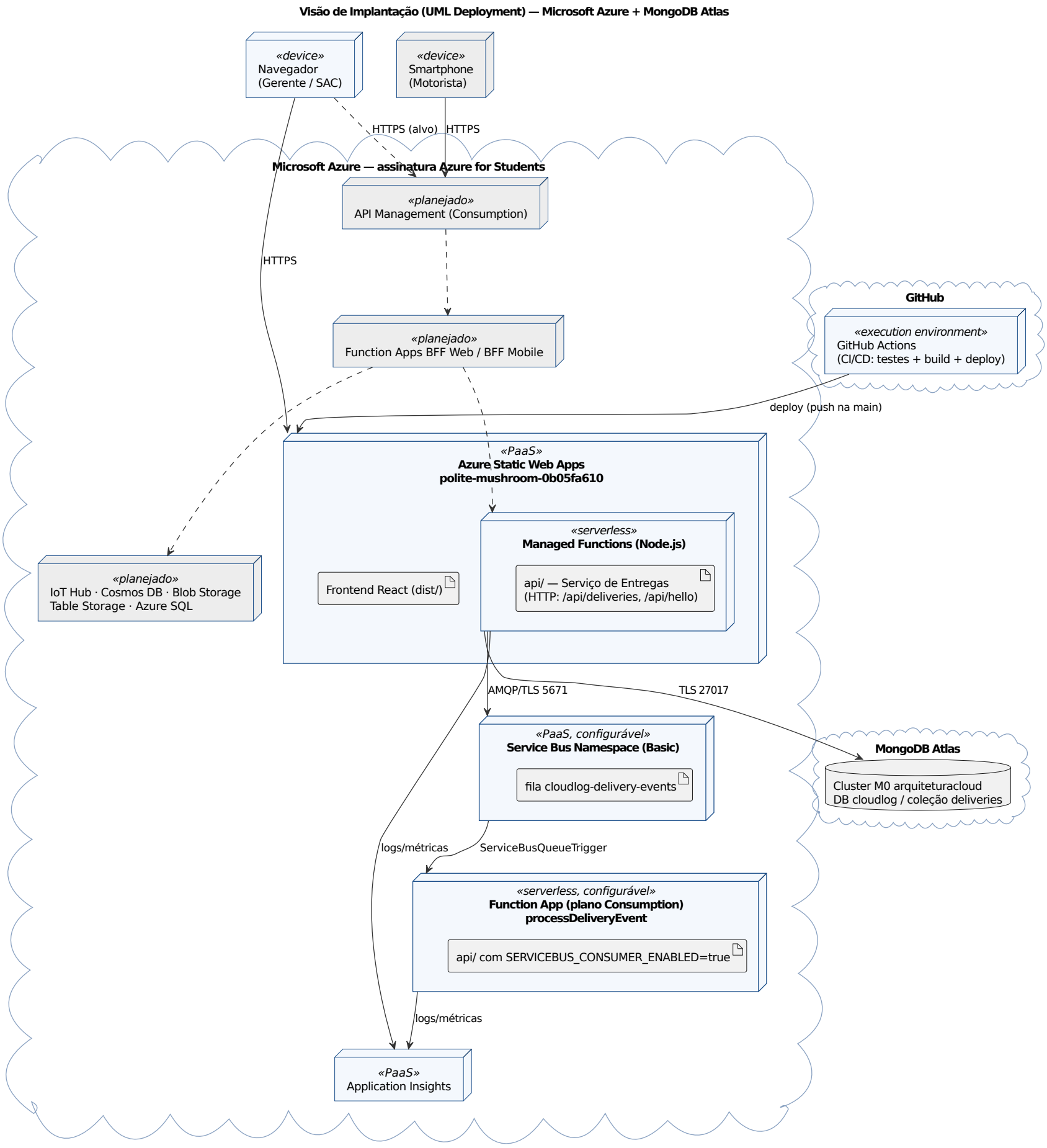


Figura 16 - Visão de implantação do CloudLog no Azure e no MongoDB Atlas.

8. Conceitos Transversais

Conceitos transversais são mecanismos ou regras que afetam várias partes da arquitetura e devem ser tratados de forma consistente.

Conceito	Diretriz arquitetural	Situação no código
Segurança	MFA, RBAC, princípio do menor privilégio, TLS em trânsito, proteção de segredos, auditoria de ações sensíveis.	TLS e segredos em variáveis da plataforma **Implementado**; MFA/RBAC **Planejado**
Identidade e sessão	Tokens de curta duração emitidos pelo Provedor de Identidade e validados no API Gateway.	**Planejado**
Observabilidade	Logs estruturados com correlation-id, métricas técnicas e de negócio, traces distribuídos e alertas operacionais.	Logs de erro e de eventos no Application Insights **Parcial**
Tratamento de erros	Erros de domínio separados de falhas técnicas; respostas padronizadas; detalhe técnico só no log.	errorMapper: 400 / 404 / 500 **Implementado**
Persistência	Cada serviço é dono do seu banco; conexões reaproveitadas entre execuções serverless.	MongoConnection reutilizada **Implementado**
Arquivos digitais	URLs não públicas (SAS), controle de acesso e metadados de auditoria.	**Planejado**
Integração assíncrona	Eventos versionados e imutáveis, publicados após a gravação; consumidores tolerantes e idempotentes; mensagens inválidas vão para a dead-letter queue.	DeliveryEvents + processDeliveryEvent **Implementado**
Serverless	Funções sem estado, inicialização preguiçosa e mesmo pacote para gatilhos HTTP e de fila.	**Implementado**
API	Contratos documentados, validação de entrada, regex escapada nas buscas e rate limiting no gateway.	Validação na entidade **Implementado**; rate limit **Planejado**
Mobile e conectividade	Fila local para ações pendentes, retry com backoff e feedback claro ao motorista quando estiver offline.	**Planejado**
Privacidade	Coleta mínima de dados, acesso restrito e retenção compatível com finalidade operacional e obrigações aplicáveis.	**Planejado**

8.1 Testes unitários de arquitetura

O backend possui 40 testes automatizados com o executor nativo do Node.js (`node:test`): 13 testes de arquitetura e 27 testes unitários, executados com `npm test` e no GitHub Actions. Os testes de arquitetura funcionam no estilo ArchUnit: `tests/support/dependencyGraph.js` lê os arquivos de `api/src`, extrai cada `require(...)` e monta o grafo de dependências; cada teste percorre o grafo e falha listando as violações encontradas.

#	Regra verificada (nome do teste)	Estilo protegido
1	Domínio é independente: só depende de si mesmo (nenhum pacote, framework ou camada externa)	Clean Architecture: núcleo puro
2	Casos de uso dependem apenas do domínio (sem HTTP, Azure ou MongoDB)	Clean Architecture: casos de uso só conhecem domínio e portas
3	Fatias verticais não dependem umas das outras	Vertical Slice: fatias isoladas
4	Infraestrutura não conhece features, HTTP nem bootstrap	Clean Architecture: infraestrutura aponta para dentro
5	shared/http não depende de features, infraestrutura ou bootstrap	Clean Architecture: utilitários HTTP transversais
6	Pacote 'mongodb' só é usado no adaptador de persistência Mongo	Database per Service / DIP: driver do banco confinado
7	Pacote '@azure/functions' só é usado nos arquivos *.function.js	Serverless: framework de funções confinado

#	Regra verificada (nome do teste)	Estilo protegido
8	Pacote '@azure/service-bus' só é usado no adaptador de mensageria	EDA: SDK de mensageria confinado
9	Casos de uso que alteram estado dependem da porta DeliveryEventPublisher (EDA)	EDA: toda alteração de estado publica evento
10	Somente o Composition Root (bootstrap) instancia a infraestrutura	DIP: só o Composition Root cria classes concretas
11	Nenhum pacote externo além dos declarados no package.json	Governança de dependências
12	Cada fatia vertical tem UseCase, handler e function	Vertical Slice: estrutura mínima de cada fatia
13	index.js registra todas as fatias verticais	Vertical Slice: fatias registradas no ponto de entrada

Arquivo de teste unitário	Cobertura	Testes
tests/unit/domain.test.js	Regras da entidade Delivery e do DeliveryStatus	7
tests/unit/useCases.test.js	Casos de uso com repositório e publicador em memória	7
tests/unit/handlers.test.js	Contrato HTTP usado pelo frontend e erros 400/404/500	5
tests/unit/mongoRepository.test.js	Adaptador MongoDB: id inválido e filtro de busca	2
tests/unit/events.test.js	EDA: ordem dos eventos, nenhum evento em falha, consumidor de alertas e publicador Service Bus	6

9. Decisões Arquiteturais

As decisões abaixo são registradas de forma resumida, no espírito de ADRs. Em relação à v0.1, as decisões ADR-02, ADR-04 e ADR-07 foram revistas, porque a implementação definiu a decomposição em microsserviços, o banco do Serviço de Entregas e o provedor de nuvem.

ID	Decisão	Alternativa	Justificativa	Consequência
ADR-01	Separar Web e Mobile	Uma única interface responsiva.	Perfis e contexto operacional diferem significativamente.	Dois clientes para manter, porém melhor UX e evolução independente.
ADR-02 (revista)	Microsserviços por capacidade, com Database per Service	API/Backend central única (v0.1).	Telemetria e entregas têm carga e evolução diferentes; autonomia por serviço.	Consistência eventual e mais componentes para operar.
ADR-03	Separar serviço de telemetria	Processar GPS dentro de outro serviço.	Volume e padrão de carga diferem das requisições transacionais.	Melhor isolamento e escalabilidade.
ADR-04 (revista)	MongoDB Atlas para o Serviço de Entregas	Banco relacional para todo o domínio (v0.1).	Entrega é documento autocontido; camada gratuita do Student Pack; já usado na Entrega 3.	Integridade referencial na aplicação; relacional mantido para Indicadores.
ADR-05	Object Storage para arquivos	Salvar binários no banco.	Custo, escalabilidade e ciclo de vida de objetos.	Controle de acesso e gestão de URLs/metadados.
ADR-06	Alertas e integrações por eventos no Azure Service Bus	Chamada síncrona ao provedor em toda operação.	Não acoplar a operação à disponibilidade de terceiros.	Necessidade de idempotência e monitorar dead-letter.
ADR-07 (revista)	Microsoft Azure com serviços serverless	Manter provider-neutral (v0.1).	Conta Azure for Students e atividades da disciplina já em Azure.	Cold start e limites do plano de consumo.
ADR-08	API Gateway (Azure API Management)	Clientes chamando serviços diretamente.	Porta única para segurança, limites e roteamento.	Ponto crítico de disponibilidade e custo por chamada.
ADR-09	Um BFF por canal (Web e Mobile)	API genérica para todas as telas.	Telas diferentes precisam de agregações diferentes.	Mais um contêiner por canal; risco de duplicação.
ADR-10	Microfrontends com Module Federation	SPA única.	Telas por domínio evoluem e publicam separadamente.	Dependências compartilhadas precisam de versão alinhada.
ADR-11	Clean Architecture + Vertical Slice no backend	Organização por camada técnica.	Mudanças localizadas e domínio testável sem infraestrutura.	Mais arquivos por funcionalidade.
ADR-12	Testes de arquitetura com node:test no CI	Revisão manual / biblioteca externa.	Proteger ADR-11 e ADR-06 sem novas dependências.	Regras baseadas em require precisam acompanhar novas pastas.
ADR-13	Publicação de eventos best-effort	Transactional Outbox.	Simplicidade nesta fase.	Evento pode se perder se o Service Bus falhar após a gravação (R9).
ADR-14	Consumidor da fila em Function App dedicada	Gatilho de fila no Static Web Apps.	Static Web Apps aceita apenas gatilhos HTTP.	Mesmo código com duas implantações, controladas por variável.

10. Requisitos de Qualidade

Os requisitos de qualidade abaixo são formulados como cenários mensuráveis. Eles orientam decisões e posterior validação; a documentação, por si só, não garante o atendimento desses objetivos. Na v1.0, parte dos cenários já é verificada por testes.

Qualidade	Cenário	Métrica / critério
Disponibilidade	Durante horário operacional, se uma instância falhar, o serviço deve continuar disponível ou recuperar-se automaticamente.	Disponibilidade mensal >= 99,9%; recuperação de instância sem intervenção manual.
Disponibilidade	O Service Bus fica indisponível durante uma inserção de entrega.	A API responde 201 e registra a falha (verificado por teste).
Desempenho	Ao receber telemetria válida, a posição deve ficar disponível para consulta rapidamente.	95% dos eventos disponíveis em até 10 s em carga nominal.
Segurança	Ao tentar acessar uma função incompatível com o perfil, o usuário deve ter acesso negado e o evento auditado.	100% das rotas protegidas por autorização no gateway; ações sensíveis auditáveis.
Usabilidade	Durante uma entrega, o motorista deve registrar comprovante com fluxo curto.	Fluxo principal em até 5 interações após abrir a entrega.
Escalabilidade	Com crescimento de veículos/eventos, a ingestão deve escalar sem duplicar toda a aplicação.	Telemetria escalável independentemente do Serviço de Entregas.
Manutenibilidade	Uma nova funcionalidade (ex.: atribuir entrega a motorista) deve ser criada sem alterar as fatias existentes.	Nova pasta em features/; testes de arquitetura verdes.
Testabilidade	Casos de uso devem ser testados sem banco e sem Azure.	27 testes unitários com adaptadores em memória.
Recuperabilidade	Em falha do banco, dados transacionais devem poder ser restaurados.	Backups automatizados do MongoDB Atlas; RPO de 24 h no plano gratuito.
Auditabilidade	Comprovantes e alterações relevantes devem possuir autoria e timestamp.	Eventos com occurredAt e aggregateId disponíveis para investigação.

10.1 Rastreabilidade entre requisitos e arquitetura

Requisito	Qualidades associadas	Elementos/decisões arquiteturais	Evidência no código
RF-01	Segurança, Auditabilidade	Provedor de Identidade; API Gateway; BFFs; TLS.	— (planejado)
RF-02	Desempenho, Escalabilidade	IoT Hub; Serviço de Telemetria; Cosmos DB; eventos.	— (planejado)
RF-03	Disponibilidade, Manutenibilidade	Serviço de Entregas; MongoDB Atlas; MFE Entregas; ADR-11.	Fatias create/search/update/deleteDelivery e testes
RF-04	Segurança, Usabilidade, Auditabilidade	Serviço de Comprovantes; Blob Storage; BFF Mobile.	— (planejado)
RF-05	Disponibilidade, Escalabilidade	Service Bus; Serviço de Alertas; ADR-06.	DeliveryEvents, processDeliveryEvent e testes de eventos
RF-06	Desempenho, Manutenibilidade	BFF Web; Serviço de Indicadores; MFE Operação.	Tela Operação (dados de demonstração)

11. Riscos e Dívidas Técnicas

A classificação abaixo é qualitativa e poderá ser substituída por uma matriz quantitativa quando houver dados de carga, custo e operação. Os riscos R5 e R8 da v0.1 foram encerrados com a escolha do provedor; os itens R9 a R14 são novos.

ID	Risco / dívida	Prob.	Impacto	Efeito	Mitigação proposta
R1	Conectividade móvel instável	Alta	Alta	Atualização/comprovante pode falhar em campo.	Suporte offline parcial, fila local e retry idempotente.
R2	Volume crescente de telemetria	Alta	Alta	Sobrecarga de ingestão, armazenamento e consulta.	Serviço dedicado, IoT Hub e política de retenção.
R3	Indisponibilidade do serviço externo de notificações	Média	Média	Alertas podem atrasar.	Processamento assíncrono, retry, DLQ e monitoramento.
R4	Exposição indevida de comprovantes	Média	Alta	Risco de privacidade e segurança.	Blob privado, autorização e URLs SAS temporárias.
R6	Custos variáveis de cloud	Média	Alta	APIM, IoT Hub e Cosmos DB podem gerar custo fora da conta estudante.	Camadas de consumo/gratuitas, budgets e exclusão de recursos após avaliação.
R7	Modelo de alertas incompleto	Média	Média	Falsos positivos/negativos afetam operação.	Regras configuráveis e testes com dados históricos.
R9	Evento perdido sem Transactional Outbox	Baixa	Alta	Alerta ou indicador não é atualizado.	Outbox no MongoDB ou Change Streams.
R10	Rotas da API ainda sem autenticação	Alta	Alta	Acesso indevido aos dados de entregas.	API Gateway + Entra External ID (RF-01) antes de dados reais.
R11	Cluster MongoDB Atlas M0 pausa por inatividade	Alta	Média	Erro 500 na API até reativar (ocorreu em outubro de 2026).	Monitorar disponibilidade; plano pago em produção.
R12	Cold start no plano de consumo	Média	Baixa	Primeira resposta mais lenta.	Flex Consumption ou instâncias pré-aquecidas.
R13	Complexidade operacional de microsserviços com equipe pequena	Média	Média	Entrega mais lenta e mais pontos de falha.	Extrair serviços aos poucos, usando o Serviço de Entregas como modelo.
R14	Frontend ainda em SPA única	Alta	Baixa	Microfrontends ainda não publicáveis separadamente.	Separar Shell e MFEs com Module Federation.